te Notes for MCQ & Viva

TABLE OF CONTENTS

Unit 1 — ER Model
Entity Sets, Attributes, Relationships, ER Diagrams, Keys, Mapping

Unit 2 — Relational Model
Relational Algebra, Calculus, Integrity Constraints, Keys

Unit 3 — SQL
DDL/DML/DCL, Joins, Nested Queries, Aggregates, Views, Triggers

Unit 4 — Functional Dependencies
Definitions, Closures, Armstrong's Axioms

Unit 5 — Normalization
1NF–BCNF, Lossless Join, Dependency Preservation

Unit 6 — Transactions
ACID Properties, Schedules, Serializability

Unit 7 — Concurrency Control
2PL, Deadlocks, Timestamp Ordering

Unit 8 — Database Recovery
Log-based, Checkpoints, Shadow Paging

Unit 9 — Indexing and Hashing
B Trees, B+ Trees, Static & Dynamic Hashing

Unit 10 — File Organization
Heap, Sequential, Index Sequential Files

Unit 11 — Query Processing & Optimization
Evaluation, Optimization Techniques

UNIT 1 — ER MODEL

1.1 What is the ER Model?

The Entity-Relationship (ER) Model is a high-level conceptual data model used to describe the structure of a database. It represents real-world objects (entities), their properties (attributes), and the associations between them (relationships). It was proposed by Peter Chen in 1976.

Entity & Entity Set

Entity Types

- Entity: A real-world object distinguishable from others (e.g., a Student, a Course).
- Entity Set: Collection of similar entities (e.g., all Students in a university).
- Strong Entity: Has its own primary key (e.g., Student with Roll_No).
- Weak Entity: Cannot be uniquely identified without a related strong entity (e.g., Dependent of Employee). Has a partial key (discriminator).

Attributes

Attribute Type	Description	Example
Simple	Cannot be divided further	Age, Roll_No
Composite	Can be divided into sub-parts	Name → First, Last
Single-valued	Holds one value	Date of Birth
Multi-valued	Holds multiple values	Phone_Numbers
Derived	Computed from other attributes	Age from DOB
Key Attribute	Uniquely identifies entity	Roll_No (underlined)
Null	Value unknown or not applicable	Middle_Name

Relationships

Relationship Concepts

- Relationship Set: Collection of similar relationships among entities.
- Degree: Unary (1 entity), Binary (2 entities — most common), Ternary (3 entities).
- Descriptive Attribute: Attribute belonging to a relationship (e.g., Grade in Student-Course Enrolls).
- Recursive Relationship: An entity related to itself (e.g., Employee Manages Employee).

Cardinality Ratios

Type	Meaning	Example
1:1 (One-to-One)	One entity in A relates to at most one in B	Person — Passport
1:N (One-to-Many)	One entity in A relates to many in B	Dept — Employees
N:1 (Many-to-One)	Many in A relate to one in B	Students — Advisor

Type	Meaning	Example
M:N (Many-to-Many)	Many in A relate to many in B	Students — Courses

Participation Constraints

Participation

- Total Participation (double line): Every entity in the set must participate in the relationship. (e.g., every Employee must work in a Department)
- Partial Participation (single line): Some entities may not participate. (e.g., not every Employee manages a Department)

1.2 ER Diagram Notation

Symbol	Representation
Rectangle (■)	Entity Set
Double Rectangle (■■)	Weak Entity Set
Ellipse (■)	Attribute
Double Ellipse (■■)	Multi-valued Attribute
Dashed Ellipse	Derived Attribute
Underlined in ellipse	Key Attribute
Diamond (■)	Relationship Set
Double Diamond (■■)	Identifying Relationship (for weak entity)
Line	Link between entity and relationship
Arrow (→)	One side of 1:1 or 1:N relationship

1.3 Keys in ER Model

Types of Keys

- Super Key: Any set of attributes that uniquely identifies an entity.
- Candidate Key: Minimal super key (no redundant attributes).
- Primary Key: Chosen candidate key used to uniquely identify entities.
- Partial Key: Discriminator for weak entity — unique only within related strong entity.
- Foreign Key: Attribute in one relation referencing the primary key of another.

1.4 Mapping ER to Relational Model

ER Construct	Relational Mapping Rule
Strong Entity Set	Create a table; all attributes become columns; primary key is key attribute.
Weak Entity Set	Create table; include primary key of owner entity as foreign key; combined PK = owner PK + partial key.
1:1 Relationship	Merge into one table OR add FK in either entity's table.

ER Construct	Relational Mapping Rule
1:N Relationship	Add FK of the '1' side into the 'N' side table.
M:N Relationship	Create a separate relationship table with FKs of both entities.
Multi-valued Attribute	Create a separate table with entity's PK + attribute value.
Composite Attribute	Include only leaf-level simple attributes in table.
Derived Attribute	Usually not stored; computed when needed.

■ *MCQ Tip: Weak entity table always includes owner entity's PK as part of its composite PK.*

■ *Viva Tip: Explain difference between participation constraint and cardinality ratio — both define relationship behavior from different perspectives.*

UNIT 2 — RELATIONAL MODEL

2.1 Relational Concepts

Term	Meaning
Relation	A table with rows and columns
Tuple	A single row in the table
Attribute	A column; has a domain (allowed values)
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)
Schema	R(A1, A2, ..., An) — structure definition
Instance	Actual data in the table at a given time

2.2 Keys

Key Types — Memorise These!

- Super Key: Set of attributes uniquely identifying every tuple.
- Candidate Key: Minimal super key (irreducible).
- Primary Key (PK): Selected candidate key; no NULL values allowed.
- Alternate Key: Candidate keys not chosen as PK.
- Foreign Key (FK): Attribute referencing PK of another table — enforces referential integrity.
- Composite Key: PK consisting of multiple attributes.

2.3 Relational Algebra

Relational Algebra is a procedural query language with operators that take relations as input and return a relation as output.

Operation	Symbol	Description	Example
Select	σ	Filters rows based on condition	σ(age>20)(Student)
Project	π	Picks specific columns	π(name, age)(Student)
Union	∪	All tuples in R or S (no duplicates); schemas must match	R ∪ S
Intersection	∩	Tuples in both R and S	R ∩ S
Difference	–	Tuples in R but not in S	R – S
Cartesian Product	×	Combines every tuple of R with every tuple of S	R × S
Rename	ρ	Renames relation or attributes	ρ(S, R)

Operation	Symbol	Description	Example
Natural Join	■	Joins on common attributes, removes duplicate columns	R ■ S
Theta Join	■θ	Join with arbitrary condition θ	R ■(R.id=S.id) S
Equi-Join	■=	Theta join where condition is equality	R ■(R.id=S.id) S
Left Outer Join	■	All tuples of left + matching right (NULLs for non-match)	R ■ S
Right Outer Join	■	All tuples of right + matching left	R ■ S
Full Outer Join	■	All tuples from both, NULLs where no match	R ■ S
Division	÷	R÷S returns tuples in R related to all tuples in S	R ÷ S

■ *SELECT (σ) operates on ROWS; PROJECT (π) operates on COLUMNS.*

2.4 Relational Calculus

Relational Calculus

- Non-procedural (declarative) — says WHAT to retrieve, not HOW.
- Tuple Relational Calculus (TRC): Variables represent tuples. {t | P(t)} — set of tuples t satisfying predicate P.
- Domain Relational Calculus (DRC): Variables represent attribute values. { | P(x1,...,xn)}.
- Both TRC and DRC are equivalent in expressive power to Relational Algebra.
- Safe Expression: Result is finite; no unbounded domains. Always assumed in practice.

2.5 Integrity Constraints

Constraint	Rule
Domain Constraint	Every attribute value must belong to its defined domain.
Key Constraint	No two tuples can have the same PK value; PK cannot be NULL.
Entity Integrity	Primary key attributes must never be NULL.
Referential Integrity	FK value must match an existing PK value or be NULL.
Semantic Integrity	Business rules (e.g., salary > 0); enforced via triggers/constraints.

UNIT 3 — SQL

3.1 SQL Command Categories

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify schema structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Manipulate data in tables
DCL	Data Control Language	GRANT, REVOKE	Control access permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

3.2 DDL Examples

```
CREATE TABLE Student(Roll INT PRIMARY KEY, Name VARCHAR(50), Age INT CHECK(Age>0), Dept VARCHAR(20));
ALTER TABLE Student ADD COLUMN Email VARCHAR(100);
DROP TABLE Student; -- deletes table + data
TRUNCATE TABLE Student; -- deletes all data, keeps structure
```

3.3 DML — SELECT Query Structure

```
SELECT [DISTINCT] columns FROM table [JOIN ...] [WHERE condition] [GROUP BY cols] [HAVING condition] [ORDER BY col [ASC|DESC]] [LIMIT n];
```

■ *Clause execution order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT*

3.4 Joins

Join Type	Returns	SQL Syntax
INNER JOIN	Only matching rows from both tables	FROM A INNER JOIN B ON A.id = B.id
LEFT OUTER JOIN	All rows from left + matching from right (NULL if no match)	FROM A LEFT JOIN B ON A.id = B.id
RIGHT OUTER JOIN	All rows from right + matching from left	FROM A RIGHT JOIN B ON A.id = B.id
FULL OUTER JOIN	All rows from both tables	FROM A FULL OUTER JOIN B ON A.id = B.id
CROSS JOIN	Cartesian product of both tables	FROM A CROSS JOIN B
SELF JOIN	Table joined with itself	FROM A a1 JOIN A a2 ON a1.mgr = a2.id
NATURAL JOIN	Joins on columns with same name	FROM A NATURAL JOIN B

3.5 Nested Queries (Subqueries)

Subquery Types

- Correlated Subquery: Inner query depends on outer query; executes once per outer row.
- Non-correlated Subquery: Inner query is independent; executes once.
- IN / NOT IN: Check if value exists in subquery result set.
- EXISTS / NOT EXISTS: Returns TRUE if subquery returns at least one row.
- ALL / ANY (SOME): Compare value against all/any values in subquery result.
- Scalar Subquery: Returns a single value used in SELECT or WHERE.

```
SELECT name FROM Student WHERE Roll IN (SELECT Roll FROM Enroll WHERE Course='DBMS');
```

3.6 Aggregate Functions

Function	Description	Example
COUNT(*)	Counts all rows	SELECT COUNT(*) FROM Student
COUNT(col)	Counts non-NULL values	SELECT COUNT(Dept) FROM Student
SUM(col)	Sum of numeric column	SELECT SUM(Salary) FROM Emp
AVG(col)	Average of numeric column	SELECT AVG(Age) FROM Student
MAX(col)	Maximum value	SELECT MAX(Marks) FROM Exam
MIN(col)	Minimum value	SELECT MIN(Marks) FROM Exam
GROUP BY	Groups rows by column for aggregation	GROUP BY Dept
HAVING	Filters groups (like WHERE for aggregates)	HAVING AVG(Marks) > 60

■ *WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation. You cannot use aggregate functions in WHERE.*

3.7 Views

Views

- A view is a virtual table based on a SELECT query. It does not store data itself.
- CREATE VIEW v AS SELECT name, dept FROM Student WHERE age > 18;
- Updatable View: A view is updatable if it maps to exactly one base table, has no GROUP BY/DISTINCT/aggregate, and includes the PK of the base table.
- Materialized View: A view that physically stores data; must be refreshed periodically.
- DROP VIEW v; — removes the view definition.

3.8 Triggers

Triggers — Key Concepts

- A trigger is a stored procedure that automatically fires on INSERT, UPDATE, or DELETE.
- BEFORE trigger: Executes before the DML operation.
- AFTER trigger: Executes after the DML operation.
- INSTEAD OF trigger: Used on views; replaces the DML operation.
- Row-level trigger (FOR EACH ROW): Fires once per affected row.
- Statement-level trigger: Fires once per DML statement, regardless of rows affected.

UNIT 4 — FUNCTIONAL DEPENDENCIES

4.1 Definition

A Functional Dependency (FD) X → Y holds in relation R if for every pair of tuples t1 and t2 in R, whenever t1[X] = t2[X], then t1[Y] = t2[Y]. This means X functionally determines Y — knowing X uniquely determines Y.

FD Types

- Trivial FD: X → Y is trivial if Y ⊆ X. (e.g., {A,B} → A)
- Non-trivial FD: Y ⊄ X. (e.g., Roll → Name)
- Completely non-trivial: X ∩ Y = ∅.
- Partial Dependency: X → Y where X is a proper subset of a candidate key (causes 2NF violations).
- Transitive Dependency: X → Y and Y → Z, where Y is not a key (causes 3NF violations).

4.2 Armstrong's Axioms

Armstrong's Axioms are a complete and sound set of inference rules for deriving all FDs from a given set F.

Axiom	Rule	Formal Statement
Reflexivity	If Y ⊆ X, then X → Y (trivial FDs)	Y ⊆ X ■ X → Y
Augmentation	If X → Y, then XZ → YZ for any Z	X → Y ■ XZ → YZ
Transitivity	If X → Y and Y → Z, then X → Z	X→Y, Y→Z ■ X→Z

Derived Rules (from Armstrong's Axioms):

Derived Rules

- Union: If X→Y and X→Z then X→YZ
- Decomposition: If X→YZ then X→Y and X→Z
- Pseudo-transitivity: If X→Y and WY→Z then WX→Z
- Composition: If X→Y and Z→W then XZ→YW

4.3 Closure of Attributes

The closure of attribute set X under FD set F, written X^+, is the set of all attributes that can be functionally determined by X using F.

Attribute Closure — Algorithm & Uses

- Algorithm: Start with result = X. For each FD A→B in F, if A ⊆ result, add B to result. Repeat until no change.
- Use X+ to check if X is a super key: X is a super key if X+ contains all attributes.
- Use X+ to check if FD X→Y holds: it holds if Y ⊆ X+.
- Canonical Cover (Fc): Minimal set of FDs equivalent to F. Remove extraneous attributes and redundant FDs.

■ *Example: Given R(A,B,C,D), F={A→B, B→C, A→D}. Compute A+: Start {A} → add B (A→B) → add C (B→C) → add D (A→D). So A+ = {A,B,C,D}. Since A+ = all attributes, A is a super key (and candidate key).*

UNIT 5 — NORMALIZATION

5.1 Why Normalize?

Database Anomalies

- Insertion Anomaly: Cannot insert data without unrelated data (e.g., can't add a course without a student).
- Deletion Anomaly: Deleting a record inadvertently deletes other needed information.
- Update Anomaly: Updating one record requires updating many duplicate copies.
- Goal: Eliminate redundancy and anomalies by decomposing tables using FD theory.

5.2 Normal Forms

Normal Form	Condition to Satisfy	Violation Example
1NF	All attributes are atomic (no multi-valued or composite attributes in a cell). Each column holds only indivisible values.	Phone: '9999,8888' in one cell
2NF	Is in 1NF + No partial dependency (non-key attribute must depend on the whole primary key, not part of it).	In key (Roll,Course)→Grade, Name depends only on Roll
3NF	Is in 2NF + No transitive dependency (non-prime attribute must not depend on another non-prime attribute).	Roll→Dept→HOD (transitive: HOD depends on Dept, not directly on key)
BCNF	Is in 3NF + For every non-trivial FD X→Y, X must be a super key. Stricter than 3NF.	In (Student,Course)→Instructor, if Instructor→Course, violates BCNF if Instructor not a super key
4NF	Is in BCNF + No multi-valued dependencies (except trivial ones).	Student →→ Hobby; Student →→ Language (independent MVDs)

■ **Memory trick: 1NF=Atomic, 2NF=No partial, 3NF=No transitive, BCNF=Every determinant is a super key.**

5.3 Decomposition Properties

Critical Properties

- Lossless Join Decomposition: When tables R1 and R2 are joined back, we recover the original relation R exactly — no spurious (extra) tuples.
- Test (Binary decomposition R→R1,R2): Lossless iff (R1∩R2) → R1 OR (R1∩R2) → R2 holds in F.
- Dependency Preservation: Every FD in F can be checked in some Ri without performing a join. If an FD must be checked across tables, preservation is lost.
- Key insight: BCNF always guarantees lossless join but may NOT preserve dependencies. 3NF decomposition always preserves dependencies AND can be made lossless.

■ *BCNF vs 3NF trade-off: BCNF is stricter but may lose FD preservation. 3NF is slightly weaker but always dependency-preserving. Always prefer 3NF when dependency preservation is needed.*

UNIT 6 — TRANSACTIONS

6.1 Transaction Concept

A transaction is a logical unit of work consisting of one or more database operations (reads/writes) that must be executed atomically — either all operations complete or none do. Transactions move the database from one consistent state to another.

States of a Transaction:

State	Meaning
Active	Transaction is being executed
Partially Committed	Last operation executed; results in main memory, not yet written to disk
Committed	All operations successfully completed and written to disk
Failed	Normal execution cannot proceed due to an error
Aborted	Transaction rolled back; database restored to pre-transaction state
Terminated	Transaction completed (committed or aborted)

6.2 ACID Properties

Property	Full Meaning	Explanation
Atomicity	All or Nothing	Either ALL operations of a transaction execute, or NONE do. Ensured by rollback on failure.
Consistency	Valid State to Valid State	A transaction brings the DB from one consistent state to another; integrity constraints maintained.
Isolation	Concurrent transactions don't interfere	Intermediate state of one transaction is not visible to other concurrent transactions.
Durability	Committed changes are permanent	Once committed, changes survive system failures (crashes, power loss). Ensured by logs.

■ **Mnemonic: ACID — like a chemical that either reacts completely or not at all, leaving the system stable.**

6.3 Schedules

Schedule Type	Definition
Serial Schedule	Transactions execute one after another with no interleaving. Always consistent but poor performance.
Non-serial Schedule	Operations of multiple transactions are interleaved. Better performance but may cause inconsistency.
Serializable Schedule	A non-serial schedule equivalent to some serial schedule. Gold standard for correctness.

Schedule Type	Definition
Conflict Serializable	Can be converted to serial by swapping non-conflicting operations.
View Serializable	Produces same view as some serial schedule (broader than conflict serializable).

6.4 Conflict & View Serializability

Key Serializability Concepts

- Conflicting Operations: Two operations conflict if they are from different transactions, operate on the same data item, and at least one is a write.
- Conflict Serializability Test: Draw a Precedence Graph (Serialization Graph). Add edge Ti→Tj if an operation in Ti conflicts with and precedes an operation in Tj. If the graph has NO CYCLE, the schedule is conflict serializable.
- View Equivalence: S1 and S2 are view equivalent if: same initial reads, same read-from relationships, same final writes.
- View Serializable ⊇ Conflict Serializable: Every conflict-serializable schedule is view-serializable, but not vice versa.
- Recoverable Schedule: A transaction Tj that reads a value written by Ti must commit AFTER Ti commits.
- Cascadeless Schedule: Transactions only read committed values — avoids cascading rollbacks.

UNIT 7 — CONCURRENCY CONTROL

7.1 Why Concurrency Control?

Concurrency Problems

- Lost Update Problem: Two transactions read and update the same value; one update is overwritten.
- Dirty Read (Temporary Update): Transaction reads a value written by an uncommitted transaction that later rolls back.
- Unrepeatable Read: Transaction reads same data twice but gets different values because another transaction modified it in between.
- Phantom Read: Transaction re-executes a query and finds new rows inserted by another committed transaction.

7.2 Lock-Based Protocols

Lock Type	Symbol	Compatibility	Description
Shared Lock (Read)	S / rl	Compatible with other S locks; not with X	Multiple transactions can read simultaneously
Exclusive Lock (Write)	X / wl	Not compatible with S or X locks	Only one transaction can write; blocks all others

Two-Phase Locking (2PL)

2PL Variants

- Growing Phase: Transaction acquires locks; cannot release any lock.
- Shrinking Phase: Transaction releases locks; cannot acquire new locks.
- Lock Point: The point where a transaction reaches maximum locks (between phases).
- Guarantee: 2PL guarantees conflict serializability.
- Strict 2PL: Holds all exclusive locks until COMMIT/ABORT. Prevents dirty reads.
- Rigorous 2PL: Holds ALL locks (S and X) until COMMIT/ABORT. Simplest, most conservative.
- Conservative 2PL: Acquires all locks before starting. No deadlocks but low concurrency.

7.3 Deadlocks

Deadlock Management

- Deadlock: A set of transactions each waiting for a lock held by another in the set — circular wait.
- Necessary conditions (Coffman): Mutual exclusion, Hold and Wait, No preemption, Circular wait.
- Detection: Maintain a wait-for graph (WFG); if a cycle exists → deadlock. Victim selection: rollback the youngest or cheapest transaction.
- Prevention — Wait-Die (non-preemptive): Older transaction waits; younger is killed (dies).
- Prevention — Wound-Wait (preemptive): Older transaction wounds (aborts) younger; younger waits.
- Prevention — Timeout: Transaction rolled back if it waits too long.
- Prevention — No Hold & Wait: Acquire all locks at once before starting.

7.4 Timestamp Ordering Protocol

Timestamp Protocol Rules

- Each transaction Ti gets a unique timestamp TS(Ti) when it starts. Older transaction has smaller timestamp.
- Thomas Write Rule: If TS(Ti) < W-timestamp(Q), ignore the write (do not abort); allows more schedules.
- Read Rule: If TS(Ti) < W-timestamp(Q) → abort Ti (it's trying to read a value already overwritten by a newer transaction).
- Write Rule: If TS(Ti) < R-timestamp(Q) → abort Ti (a newer transaction already read the old value).
- Guarantees serializability without locks — deadlock-free but may cause starvation (same transaction keeps aborting).

UNIT 8 — DATABASE RECOVERY

8.1 Failure Types

Failure Type	Description	Recovery Method
Transaction Failure	Logical error or system crash during transaction	Rollback (UNDO)
System Crash	Power failure, hardware/software crash; volatile memory lost	Redo committed, Undo uncommitted
Disk Failure	Physical disk damage; data lost	Restore from backup + apply logs

8.2 Log-Based Recovery

Log-Based Recovery

- Write-Ahead Logging (WAL): Log record must be written to stable storage BEFORE the corresponding database change is written to disk.
- Log Record Format: for updates; , , .
- Undo Log: Stores old values; used to undo uncommitted transactions. Rule: Write log before writing DB; write COMMIT only after all changes written.
- Redo Log: Stores new values; used to redo committed transactions. Rule: Write log before DB; write COMMIT to log before writing changes to disk.
- Undo-Redo Log: Stores both old and new values; most flexible. Rule: Write log before DB.
- REDO: Re-applies changes of committed transactions (in case they weren't written to disk).
- UNDO: Reverses changes of uncommitted/aborted transactions.

8.3 Checkpoints

Checkpoints

- Problem: During recovery, scanning the entire log is expensive.
- Checkpoint: A periodic snapshot where all dirty buffers are flushed to disk and a record is written to log.
- Recovery with checkpoint: Scan log from last checkpoint only — ignore transactions committed before checkpoint (already on disk).
- Fuzzy Checkpoint: Allows transactions to continue during checkpointing; more efficient but slightly complex.
- ARIES Protocol (industry standard): Uses LSN (Log Sequence Number), supports fuzzy checkpoints, uses Analysis-Redo-Undo phases.

8.4 Shadow Paging

Shadow Paging

- Maintains two page tables: current (active) and shadow (backup of last committed state).
- On COMMIT: Current page table becomes the new shadow; old shadow discarded.
- On ABORT: Current page table is discarded; shadow page table is restored — instant rollback.
- Advantage: No need for undo log; simple recovery.
- Disadvantage: Fragmentation of database; high overhead for large databases; does not support concurrent transactions well.
- Used in: CouchDB (historically), some embedded databases.

UNIT 9 — INDEXING AND HASHING

9.1 Indexing Concepts

Index Types

- Index: Data structure to speed up data retrieval without scanning entire table.
- Search Key: Attribute(s) used to look up records in an index.
- Dense Index: Index entry for every search-key value in the file.
- Sparse Index: Index entry for only some values (e.g., one per block). Requires ordered file.
- Primary Index: Built on ordering key field of an ordered data file.
- Secondary Index (Non-clustering): Built on non-ordering field; always dense.
- Clustered Index: Data file is physically ordered by the index key.
- Non-clustered Index: Data file order is independent of index order.
- Multi-level Index: Index on an index to reduce I/O (outer index is sparse, inner is primary).

9.2 B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data storage	Data pointers in ALL nodes (internal + leaf)	Data pointers ONLY in leaf nodes
Internal nodes	Store both keys and data pointers	Store only keys (routing keys)
Leaf nodes	Not necessarily linked	Linked as a doubly linked list
Sequential access	Requires tree traversal (slower)	Very fast via leaf linked list
Space	Less key repetition; more compact	Keys may repeat in internal nodes
Search	May stop at internal node if key found	Always traverses to leaf
Preferred in	File systems (e.g., B-tree variants)	RDBMS indexes (MySQL InnoDB, PostgreSQL)

■ *B+ Trees are preferred in databases because range queries are fast (traverse leaf linked list) and leaf nodes have uniform access time.*

B+ Tree Properties

- Order n B+ Tree: Each internal node has between ■n/2■ and n children; root has between 2 and n children.
- Each internal node with k children has k-1 search keys.
- All leaf nodes are at the same level (balanced).
- Insertion: Insert at leaf; split if overflow; propagate split upward if needed.
- Deletion: Remove from leaf; redistribute or merge if underflow.

9.3 Hashing

Type	Description	Pros/Cons
Static Hashing	Fixed number of buckets h(k) = k mod M. Overflow handled by chaining.	Simple; but performance degrades with growth
Dynamic Hashing (Extendible)	Directory of pointers to buckets; doubles when bucket overflows. Uses global/local depth.	Handles growth; but directory can become large
Linear Hashing	Splits buckets linearly using a split pointer; no directory needed.	Space-efficient; gradual splitting

■ *Hashing is best for equality queries (exact match). B+ Trees are better for range queries and ordered access.*

UNIT 10 — FILE ORGANIZATION

10.1 Overview

File organization determines how records are physically arranged on storage. The choice affects performance of insert, delete, search, and sequential scan operations.

Organization	Structure	Best For	Weakness
Heap File	Records stored in insertion order; no sorting. Newest records appended to last block.	Bulk loads, full scans	Slow search O(n); requires full scan
Sequential File	Records sorted by a key field. Free space reserved in each block for insertions; overflow chains used.	Range queries, ordered output	Expensive insertions; reorganization needed periodically
Index Sequential (ISAM)	Ordered data file + sparse primary index (index sequential access method). Overflow for new inserts.	Mixed read/insert workloads	Overflow chains degrade with time
Clustered File	Physically orders records by a clustering key; B+ tree often used.	Queries filtering by range on clustering key	Can only cluster on one key
Hashed File	Records stored based on hash function output. No order.	Point queries (equality search)	Poor for range queries

■ *Heap files are simplest but slowest for search. Sequential files are fast for ordered access but inserts are costly. Most RDBMS use B+ tree clustered indexes as the primary storage structure.*

UNIT 11 — QUERY PROCESSING & OPTIMIZATION

11.1 Query Processing Steps

Query Processing Pipeline

- 1. Parsing & Translation: SQL query → parse tree → relational algebra expression.
- 2. Optimization: Query optimizer generates multiple execution plans and estimates cost; selects cheapest plan.
- 3. Evaluation: Query execution engine executes the chosen plan using physical operators.

11.2 Query Evaluation

Algorithm	Description	Cost
Linear Scan (Full Table Scan)	Read every block of the relation	b_r (number of blocks)
Binary Search	Only if file is sorted on search key	■log2(b_r)■
Primary Index (equality)	Use index to find block; fetch record	h_i + 1 (tree height + 1)
Secondary Index (equality)	Index lookup + one or more record fetches	h_i + number of matches
Nested Loop Join	For each tuple in outer, scan all of inner	n_r * b_s + b_r
Block Nested Loop Join	For each block of outer, scan inner	b_r * b_s + b_r
Sort-Merge Join	Sort both relations then merge	Cost of sorting + b_r + b_s
Hash Join	Hash both relations on join key	3*(b_r + b_s) typically

11.3 Query Optimization Techniques

Optimization Approaches

- Heuristic Optimization (Rule-Based): Transform query tree using equivalence rules without cost estimation.
- Cost-Based Optimization: Estimate cost of multiple plans using statistics (tuple counts, block counts, selectivity); pick minimum.
- Key Heuristic Rules:

Heuristic Rule	Effect
Push SELECT down	Apply selections as early as possible to reduce intermediate relation size
Push PROJECT down	Project unneeded attributes early to reduce tuple width
Combine SELECT & Cartesian to JOIN	Replace σ(R × S) with R ■ S for efficiency
Perform most restrictive SELECT first	Reduces data volume earliest
Use left-deep join trees	Allows pipelining; one input always a base relation
Pipelining over Materialization	Pass output of one operation directly to next; avoids writing intermediate results to disk

11.4 Statistics Used in Cost Estimation

Key Statistics & Notation

- n_r: Number of tuples in relation r.
- b_r: Number of disk blocks of relation r.
- l_r: Size of a tuple in bytes.
- f_r: Blocking factor = ■block_size / l_r■.
- V(A, r): Number of distinct values for attribute A in r (cardinality).
- Selectivity: Fraction of tuples satisfying a condition — lower = more selective = fewer tuples.
- Histograms: Used to estimate selectivity for range queries more accurately.

■ MCQ & VIVA QUICK REFERENCE

Must-Know Facts for MCQs

Fact	Answer
ER model proposed by?	Peter Chen, 1976
Relational model proposed by?	E.F. Codd, 1970
Weakest normal form?	1NF
BCNF vs 3NF — which preserves dependencies?	3NF (not always BCNF)
Which guarantees lossless join in decomposition?	Both 3NF and BCNF can; test with R1∩R2 → R1 or R2
2PL guarantees?	Conflict serializability
ACID property ensured by recovery?	Atomicity (rollback) and Durability (logs)
WAL stands for?	Write-Ahead Logging
Best structure for range queries?	B+ Tree
Best structure for equality (exact match) queries?	Hashing
SELECT operates on rows or columns?	Rows (σ)
PROJECT operates on rows or columns?	Columns (π)
Cascadeless schedule avoids?	Cascading rollbacks (reads only committed values)
View serializable ⊇ Conflict serializable?	TRUE — every conflict serializable is view serializable
TRUNCATE vs DELETE?	TRUNCATE is DDL (cannot rollback without savepoint); DELETE is DML (can rollback)
NULL = NULL evaluates to?	UNKNOWN (not TRUE) in SQL
HAVING filters?	Groups (after GROUP BY); WHERE filters rows (before GROUP BY)
Phantom read problem occurs in?	Isolation level READ COMMITTED and below
Shadow paging replaces?	Logging — uses current and shadow page tables
Division operator (R÷S) used for?	"For all" queries (e.g., students enrolled in ALL courses)

■ *If asked about 'spurious tuples', they arise from BAD (lossy) decomposition. Always verify lossless join property.*

■ *ARIES = Analysis + Redo + Undo. It is the gold standard recovery algorithm used in most production RDBMS.*

All the best, Parth! ■